

PROYECTO INTERMODULAR



**ALONSO
DE AVELLANEDA**

Alonso de Avellaneda

Técnico Superior en Desarrollo de Aplicaciones
Multiplataforma

Clef

Memoria

Autor: Óscar Padrino López

Autor: Adrián Muñoz Saiz

Autor: José Manuel Perdices Hernández

Director: César Rodríguez Casero

Tutor curso: Diana Pastor Lazcano

Tutor proyecto: Alejandro De Acuña Jiménez

Alcalá de Henares, mayo de 2026

Clef



Memoria

Contenido

1. Introducción y motivación	3
1.1 Alcance del proyecto	4
2. Objetivos	5
2.1 Objetivo general	5
2.2 Objetivos específicos	5
2.3 Estado del arte	6
2.4 Análisis de soluciones similares	7
2.5 Justificación de las tecnologías elegidas	8
3. Requisitos del sistema	9
3.1 Requisitos funcionales	9
3.2 Requisitos no funcionales	13
3.3 Restricciones del sistema	14
4. Planificación y temporalización	15
4.1 Metodología de desarrollo	15
4.2 Fases del proyecto	15
4.3 Diagrama de Gantt	17
4.4 Puntos críticos del proyecto	18
5. Diseño del sistema y arquitectura	19
5.1 Arquitectura general del sistema	19
5.2 Modelo de datos	20
5.3 Arquitectura criptográfica: el modelo Zero-Knowledge	24
5.4 Flujos de autenticación	28
5.5 Estructura de navegación	29
5.6 Diseño de interfaz	31
6. Desarrollo	34
6.1 Entorno de desarrollo	34
6.2 Módulo de autenticación y gestión de sesión	34
6.3 Motor criptográfico	35
6.4 Gestión del vault y credenciales	36
6.5 Sistema de Autofill	37
6.6 Sincronización y resolución de conflictos	39
6.7 Notificaciones y expiración de contraseñas	39
6.8 Sistemas de correos al usuario	40
7. Seguridad	41
7.1 Modelo de cifrado	41

7.2 Protecciones implementadas.....	41
7.3 Vectores de ataque y mitigaciones.....	42
8. Software y herramientas utilizadas.....	43
9. Pruebas y validación.....	45
9.1 Estrategia de pruebas.....	45
9.2 Casos de prueba funcionales.....	45
9.3 Pruebas de seguridad.....	47
9.4 Pruebas Unitarias y Validación del Sistema.....	48
10. Despliegue.....	50
10.1 Requisitos previos.....	50
10.2 Configuración de Firebase.....	50
10.3 Compilación y ejecución.....	51
11. Memoria económica.....	52
11.1 Costes de hardware.....	52
11.2 Costes de software y licencias.....	53
11.3 Costes de desarrollo (recursos humanos).....	54
11.4 Coste total del proyecto.....	54
12. Conclusiones y mejoras futuras.....	55
12.1 Conclusiones.....	55
12.2 Dificultades encontradas.....	56
12.3 Mejoras futuras.....	57
12.4 Escalabilidad de Soporte y Ayuda al Usuario.....	58
13. Bibliografía y fuentes consultadas.....	59
14. Anexos.....	62
Anexo A.....	62
Anexo B.....	62
Anexo C.....	62

1. Introducción y motivación

Hoy en día es prácticamente imposible no acumular decenas de cuentas digitales. El correo, el banco, las redes sociales, las plataformas de streaming, el trabajo... cada servicio pide su propia contraseña, y la mayoría recomienda que sean únicas y complejas. El resultado, en la práctica, es que casi todo el mundo acaba usando la misma clave para todo o con pequeñas variaciones, lo cual convierte una brecha de seguridad en un único sitio en un problema que se propaga a todos los demás.

Los gestores de contraseñas comerciales existen para resolver este problema, pero todos ellos tienen algo en común: requieren que el usuario deposite su confianza en la empresa que los opera. Almacenan los datos cifrados en sus servidores, y aunque técnicamente no pueden leerlos, los servidores existen, pueden ser hackeados, y la empresa puede cambiar sus políticas. En 2022, LastPass —uno de los gestores más populares del mundo— sufrió una brecha que expuso los vaults cifrados de millones de usuarios. El cifrado los protegió, pero la situación demostró que depositar todos los datos en un tercero siempre implica un riesgo.

Clef nació de querer llevar ese nivel de control al usuario final. La idea central es sencilla: los datos solo se cifran y descifran en el propio dispositivo del usuario, y lo que viaja a la nube es un bloque completamente opaco. Firebase, que usamos como infraestructura, nunca ve una sola contraseña en claro. Ni nosotros como desarrolladores podríamos acceder a los datos de un usuario aunque quisiéramos, porque la clave que los descifra nunca sale del dispositivo.

El proyecto se ha desarrollado como aplicación nativa Android en Java, orientada a dispositivos con Android 9 o superior. La elección de Android no fue arbitraria: es el sistema operativo móvil mayoritario, y desarrollar en nativo nos permitía acceder directamente a las APIs de seguridad del sistema, en particular al Android Keystore y al framework de Autofill, que son pilares fundamentales de la arquitectura.

1.1 Alcance del proyecto

Clef es una aplicación completa de gestión de contraseñas que cubre el ciclo entero: registro seguro, almacenamiento cifrado, sincronización en la nube, generación de contraseñas seguras, integración con el sistema de autocompletado de Android, y recuperación de acceso. Se ha prestado especial atención a dos aspectos que en muchos proyectos académicos quedan como secundarios: la seguridad criptográfica real y la experiencia de usuario.

El alcance no incluye versión web ni versión iOS. Tampoco se ha desarrollado un backend propio; Firebase actúa como infraestructura gestionada, lo cual nos permitió concentrar el esfuerzo en la lógica de la aplicación.

2. Objetivos

2.1 Objetivo general

Desarrollar una aplicación Android de gestión de contraseñas con arquitectura Zero-Knowledge, que garantice que los datos del usuario nunca sean accesibles desde el servidor, integrando cifrado de extremo a extremo, autenticación biométrica y autocompletado automático en otras aplicaciones.

2.2 Objetivos específicos

- Implementar un sistema de cifrado robusto basado en AES-256-GCM y derivación de clave con PBKDF2, de forma que la contraseña maestra del usuario nunca se almacene ni se transmita en ningún momento.
- Diseñar una arquitectura de claves de tres capas que separe la clave de cifrado de los datos (DEK) de las claves derivadas de la contraseña del usuario (KEK), permitiendo recuperación de acceso sin comprometer la seguridad.
- Integrar el servicio de Autofill de Android para que la aplicación pueda rellenar automáticamente credenciales en otras apps, con autenticación biométrica previa.
- Garantizar el funcionamiento offline de la aplicación, sincronizando con la nube de forma transparente cuando haya conexión disponible.
- Proteger la aplicación frente a ataques de fuerza bruta con bloqueos progresivos y bloqueo permanente de cuenta si se agotan todos los intentos.
- Desarrollar un generador de contraseñas criptográficamente seguro, configurable por el usuario.

- Implementar un sistema de notificaciones que avise al usuario cuando alguna de sus contraseñas esté próxima a caducar o haya caducado.
- Construir una interfaz moderna, intuitiva y que funcione tanto en orientación vertical como horizontal.

2.3 Estado del arte

La gestión de contraseñas ha evolucionado mucho en los últimos años. Durante mucho tiempo, la solución habitual era anotar las contraseñas en papel o reutilizarlas, lo cual sigue siendo habitual entre usuarios no técnicos. Los primeros gestores de contraseñas de escritorio, como KeePass (2003), plantearon el modelo de bóveda cifrada local que todavía es válido hoy en día.

La siguiente generación, liderada por servicios como LastPass (2008) y 1Password (2006), añadió la sincronización en la nube y las aplicaciones móviles, lo que los convirtió en herramientas viables para el uso cotidiano. Sin embargo, al centralizar los datos en servidores propios, introdujeron una dependencia de terceros que ha demostrado ser problemática en varias ocasiones.

Bitwarden, lanzado en 2016 y de código abierto, supuso un paso adelante al publicar su código para auditoría pública y ofrecer la opción de autoalojamiento. Es actualmente el referente más cercano a lo que hemos implementado en términos de filosofía de seguridad.

Los fabricantes de sistemas operativos también han entrado en el sector: Google Password Manager y Apple iCloud Keychain ofrecen gestión integrada de contraseñas, pero su principal limitación es que están atados al ecosistema del fabricante y no ofrecen el nivel de control y transparencia que tiene una solución dedicada.

2. Objetivos

2.4 Análisis de soluciones similares

Solución	Plataforma	Código abierto	Zero-Knowledge	Autofill Android	Tarifa
Bitwarden	Multiplataforma	Sí	Sí	Sí	Gratuito/Premium
1Password	Multiplataforma	No	Sí	Sí	De pago
KeePass	Escritorio	Sí	Sí (local)	Plugins	Gratuito
LastPass	Multiplataforma	No	Sí	Sí	Freemium
Google Password Manager	Android / Chrome	No	No	Sí	Gratuito
Clef	Android	-	Sí	Sí	-

La diferencia principal de Clef respecto a estos servicios no está en las funcionalidades —que son comparables a las de cualquier gestor moderno— sino en que es un proyecto propio construido desde cero con comprensión plena de cada decisión criptográfica, sin depender de librerías de terceros para la lógica de cifrado central.

2.5 Justificación de las tecnologías elegidas

Android nativo con Java: La alternativa multiplataforma (Flutter, React Native) habría facilitado una eventual versión iOS, pero habría añadido una capa de abstracción entre el código y las APIs de seguridad del sistema. El acceso directo a BiometricPrompt, AutofillService y el Android Keystore justifica el enfoque nativo. Java se eligió sobre Kotlin porque es el lenguaje que el equipo domina con mayor solidez tras el ciclo.

Firebase: Montar un backend propio para un TFC habría desviado tiempo y esfuerzo del objetivo principal. Firebase ofrece autenticación robusta, base de datos en tiempo real y Cloud Functions, todo gestionado. Como los datos llegan siempre cifrados, la falta de control sobre la infraestructura no representa un riesgo para la privacidad del usuario.

AES-256-GCM + PBKDF2: Son los estándares recomendados por NIST para cifrado simétrico y derivación de contraseñas respectivamente. GCM añade autenticación integrada al cifrado, lo que permite detectar manipulaciones del texto cifrado sin necesidad de un HMAC separado. PBKDF2 con 230.000 iteraciones hace que un ataque de diccionario sea computacionalmente inviable con hardware convencional.

Material Design 3: La guía de diseño de Google para Android garantiza una experiencia de usuario consistente con el resto del sistema operativo y facilita la accesibilidad.

3. Requisitos del sistema

3.1 Requisitos funcionales

ID	Requisito	Prioridad
RF-01	El sistema permitirá el registro de nuevos usuarios mediante cuenta Google.	Alta
RF-02	El sistema permitirá el registro mediante correo electrónico y contraseña.	Alta
RF-03	En el primer acceso, el usuario deberá crear una contraseña maestra de al menos 8 caracteres.	Alta
RF-04	El sistema generará un código PUK único durante el registro, que el usuario podrá usar para recuperar el acceso.	Alta
RF-05	El sistema mostrará el código PUK al usuario una única vez, en el momento del registro, con advertencia explícita de que no podrá recuperarse.	Alta
RF-06	El usuario podrá desbloquear la aplicación introduciendo su contraseña maestra.	Alta
RF-07	El usuario podrá desbloquear la aplicación mediante autenticación biométrica (huella dactilar o reconocimiento facial), si está disponible en el dispositivo.	Alta
RF-08	La aplicación se bloqueará automáticamente tras un período de inactividad configurable por el usuario.	Alta
RF-09	El sistema verificará la contraseña maestra del usuario, como mínimo, cada 3 días, aunque la biometría esté activa.	Media

RF-10	El usuario podrá añadir nuevas credenciales con los campos: título, nombre de usuario, contraseña, URL, notas y categoría.	Alta
RF-11	El usuario podrá editar cualquier credencial almacenada.	Alta
RF-12	El usuario podrá eliminar credenciales almacenadas.	Alta
RF-13	El sistema conservará un historial de las últimas 5 contraseñas utilizadas en cada credencial.	Media
RF-14	El usuario podrá marcar credenciales como favoritas para acceder a ellas de forma más rápida.	Media
RF-15	El sistema mostrará el listado de credenciales con búsqueda en tiempo real por título y nombre de usuario.	Alta
RF-16	El usuario podrá filtrar las credenciales por categoría.	Media
RF-17	El usuario podrá filtrar el listado para mostrar únicamente las credenciales marcadas como favoritas.	Media
RF-18	Las credenciales caducadas se mostrarán en una sección diferenciada del listado principal.	Media
RF-19	El usuario podrá asignar una categoría a cada credencial entre las siguientes: Banco, Social, Trabajo, Juegos, Compras, Transporte, Ocio, Deporte y Otros.	Media
RF-20	El usuario podrá generar contraseñas aleatorias seguras desde la pantalla del generador	Alta
RF-21	El generador permitirá configurar la longitud de la contraseña (entre 4 y 128 caracteres).	Alta

3. Requisitos del sistema

RF-22	El generador permitirá activar o desactivar el uso de mayúsculas, minúsculas, números y caracteres especiales.	Alta
RF-23	El generador garantizará que la contraseña generada incluya al menos un carácter de cada tipo activo.	Alta
RF-24	La aplicación ofrecerá autocompletado de credenciales en otras aplicaciones instaladas en el dispositivo (Autofill).	Alta
RF-25	El servicio de autofill aprenderá y recordará la aplicación desde la que se usó cada credencial.	Media
RF-26	El servicio de autofill propondrá guardar nuevas credenciales detectadas en formularios de otras aplicaciones.	Media
RF-27	El vault se sincronizará automáticamente con la nube cuando haya conexión a internet.	Alta
RF-28	La aplicación funcionará sin conexión a internet, utilizando los datos guardados localmente.	Alta
RF-29	En caso de conflicto entre la versión local y la remota del vault, el sistema realizará una fusión automática conservando los cambios de ambas.	Media
RF-30	El usuario podrá recuperar el acceso a su cuenta mediante el código PUK si olvida la contraseña maestra.	Alta
RF-31	Tras una recuperación con PUK exitosa, el sistema invalidará el PUK anterior y generará uno nuevo.	Alta
RF-32	El sistema bloqueará la cuenta de forma permanente si se agotan los 5 intentos disponibles de desbloqueo o de recuperación.	Alta

RF-33	El sistema notificará al usuario cuando alguna de sus contraseñas esté próxima a caducar o haya caducado.	Media
RF-34	El período de expiración de contraseñas será configurable: 3 meses, 6 meses o 1 año.	Media
RF-35	El usuario podrá activar y desactivar la autenticación biométrica desde los ajustes de la aplicación.	Media
RF-36	El usuario podrá exportar el vault completo a un archivo JSON cifrado.	Baja
RF-37	El usuario podrá importar un vault desde un archivo JSON cifrado previamente exportado.	Baja
RF-38	La aplicación mostrará estadísticas básicas del vault (número de credenciales, categorías, contraseñas caducadas).	Baja
RF-39	El sistema mostrará la fortaleza estimada de la contraseña maestra en tiempo real durante su creación.	Media
RF-40	La aplicación copiará la contraseña al portapapeles de forma segura y la borrará automáticamente tras un período breve.	Media

3. Requisitos del sistema

3.2 Requisitos no funcionales

ID	Requisito	Categoría
RNF-01	Todo el cifrado y descifrado de datos se realizará exclusivamente en el dispositivo del usuario, nunca en el servidor.	Seguridad
RNF-02	Los datos almacenados en Firebase estarán siempre cifrados con AES-256-GCM antes de ser enviados.	Seguridad
RNF-03	La derivación de la contraseña maestra se realizará mediante PBKDF2WithHmacSHA256 con un mínimo de 230.000 iteraciones.	Seguridad
RNF-04	La clave de cifrado del vault (DEK) solo existirá en la memoria RAM del dispositivo durante la sesión activa, y será sobrescrita con ceros al bloquear la aplicación.	Seguridad
RNF-05	Las claves biométricas se almacenarán en el Android Keystore del dispositivo, sin posibilidad de exportación.	Seguridad
RNF-06	El código en versión release estará ofuscado mediante ProGuard/R8.	Seguridad
RNF-07	La aplicación detectará si el dispositivo tiene root y restringirá las operaciones sensibles en consecuencia.	Seguridad
RNF-08	La aplicación será compatible con dispositivos Android 9 (API 28) o superior	Compatibilidad
RNF-09	Las operaciones criptográficas se ejecutarán en hilos secundarios para no bloquear la interfaz de usuario.	Rendimiento

RNF-10	El tiempo de desbloqueo con contraseña maestra no superará los 2 segundos en condiciones normales.	Rendimiento
RNF-11	La interfaz seguirá las guías de Material Design 3 de Google.	Usabilidad
RNF-12	La aplicación funcionará correctamente en orientación vertical y horizontal.	Usabilidad
RNF-13	Los textos de la interfaz estarán en español.	Localización
RNF-14	La aplicación no almacenará contraseñas en texto plano en ningún punto del sistema, incluyendo logs y preferencias.	Seguridad
RNF-15	Las preferencias de la aplicación se almacenarán cifradas mediante EncryptedSharedPreferences.	Seguridad

3.3 Restricciones del sistema

- La recuperación de contraseñas de otros usuarios no es posible en ningún caso. La arquitectura zero-knowledge impide, por diseño, que los desarrolladores puedan acceder a los datos de cualquier cuenta.
- El código PUK se genera una única vez y no puede regenerarse sin acceso previo a la cuenta. Si un usuario pierde tanto la contraseña maestra como el PUK, no existe mecanismo de recuperación.
- La autenticación biométrica requiere que el dispositivo tenga registrada al menos una credencial biométrica fuerte (huella dactilar o reconocimiento facial con hardware dedicado). Las soluciones basadas únicamente en PIN o patrón no se consideran autenticadores fuertes a efectos de esta aplicación.
- La funcionalidad de Autofill requiere que el usuario habilite Clef como servicio de autofill en los ajustes del sistema Android.

4. Planificación y temporalización

4.1 Metodología de desarrollo

El equipo adoptó un enfoque iterativo e incremental, con ciclos cortos de desarrollo centrados en funcionalidades concretas. No se siguió ninguna metodología ágil formal como Scrum, pero sí se tomaron prestados algunos de sus principios: trabajo en bloques pequeños, revisión continua del resultado, y capacidad de ajustar prioridades en función de los problemas encontrados.

La comunicación entre los tres miembros del equipo se mantuvo de forma constante, con reuniones de puesta en común al inicio de cada nueva fase y validación conjunta antes de dar por cerrada cada funcionalidad. El control de versiones se gestionó con Git, usando la rama “main” como rama de integración estable.

4.2 Fases del proyecto

El proyecto se desarrolló entre el 1 de febrero de 2026 y el 15 de mayo de 2026, un total de aproximadamente 14 semanas, distribuidas en las siguientes fases:

Fase 1 — Análisis y diseño inicial (semanas 1-2 | 1-14 feb)

Definición del alcance del proyecto, análisis de soluciones existentes e investigación sobre los estándares criptográficos a utilizar. Diseño de la arquitectura de claves y bocetos de la interfaz principal. Configuración del entorno de desarrollo y del proyecto en Firebase.

Fase 2 — Autenticación y base del sistema (semanas 3-4 | 15-28 feb)

Implementación del flujo de registro e inicio de sesión con Google y email/contraseña a través de Firebase Auth. Creación de la contraseña maestra, generación del PUK y persistencia inicial en Firestore. Primera versión funcional del motor criptográfico (CryptoUtils y KeyManager).

Fase 3 — Vault y cifrado completo (semanas 5-7 | 1-21 mar)

Implementación del vault cifrado con el modelo de tres capas (Caja A, Caja B, DEK). Persistencia local del vault cifrado. Gestión de sesión en memoria (SessionManager). Desbloqueo con contraseña maestra, protección contra fuerza bruta (BruteForceGuard, AccountBlocker), además de detección si el dispositivo se encuentra en estado root durante el inicio de sesión. Recuperación de acceso con PUK.

Fase 4 — Interfaz principal y gestión de credenciales (semanas 8-9 | 22 mar - 4 abr)

Desarrollo de la pantalla principal con listado de credenciales, búsqueda, filtros por categoría y favoritas. Diálogo de añadir y editar credenciales con integración del generador de contraseñas. Historial de contraseñas por credencial. Integración de iconos por categoría y plataforma.

Fase 5 — Autofill y biometría (semanas 10-11 | 5-18 abr)

Implementación completa del servicio de Autofill de Android: detección de campos, ranking de credenciales, sesión transitoria para flujos multi-pantalla y guardado de nuevas credenciales. Autenticación biométrica mediante Android Keystore.

4. Planificación y temporalización

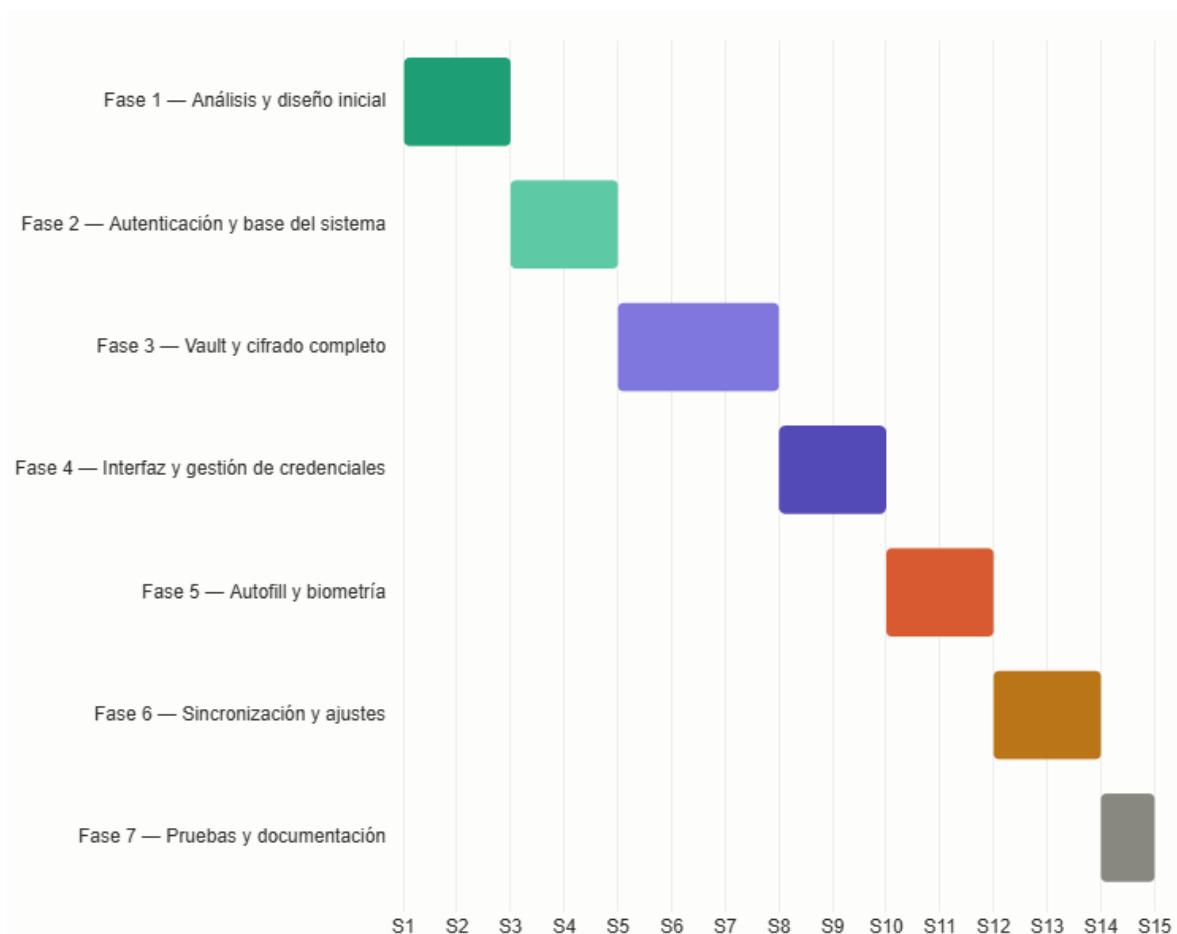
Fase 6 — Sincronización, notificaciones y ajustes (semanas 12-13 | 19 abr - 2 may)

Sincronización en tiempo real con Firestore, resolución de conflictos con optimistic locking. Sistema de expiración de contraseñas con WorkManager y notificaciones. Pantalla de ajustes completa: biometría, auto-lock, período de expiración, import/export.

Fase 7 — Pruebas, pulido y documentación (semana 14 | 3-15 may)

Pruebas funcionales, corrección de errores, ajustes de diseño, orientación horizontal, y redacción de la memoria y documentación de soporte.

4.3 Diagrama de Gantt



4.4 Puntos críticos del proyecto

El punto más crítico del proyecto fue el diseño e implementación de la arquitectura de claves. Cualquier error en ese componente podría haber comprometido la seguridad de todos los datos, o haber dejado a los usuarios sin posibilidad de recuperar su vault. Se dedicó tiempo especial a validar que cada flujo —registro, login, recuperación con PUK— fuera correcto y que las claves intermedias se zerorizaran adecuadamente tras su uso.

Un segundo punto crítico fue la integración del servicio de Autofill. Android impone restricciones considerables sobre cuándo y cómo un servicio puede rellenar formularios, y la variedad de aplicaciones con flujos de login no estándar (varios pasos, múltiples pantallas) requirió el desarrollo de la sesión transitoria para evitar que el usuario tuviera que autenticarse biométricamente dos veces en el mismo flujo.

Un tercer punto de decisión de diseño fue la detección de entorno comprometido. Bloquear el arranque en dispositivos rooteados es sencillo; el reto real es no generar falsos positivos en dispositivos de fabricante que incluyen propiedades normalmente asociadas al root (ro.debuggable, test-keys) por razones ajenas al usuario. Se diseñó un sistema de dos niveles: indicios inequívocos provocan bloqueo inmediato, mientras que los indicios ambiguos, solo bloquean al alcanzar un umbral de tres señales simultáneas, reduciendo el riesgo de impedir el uso a usuarios legítimos sin sacrificar la protección frente a entornos realmente comprometidos.

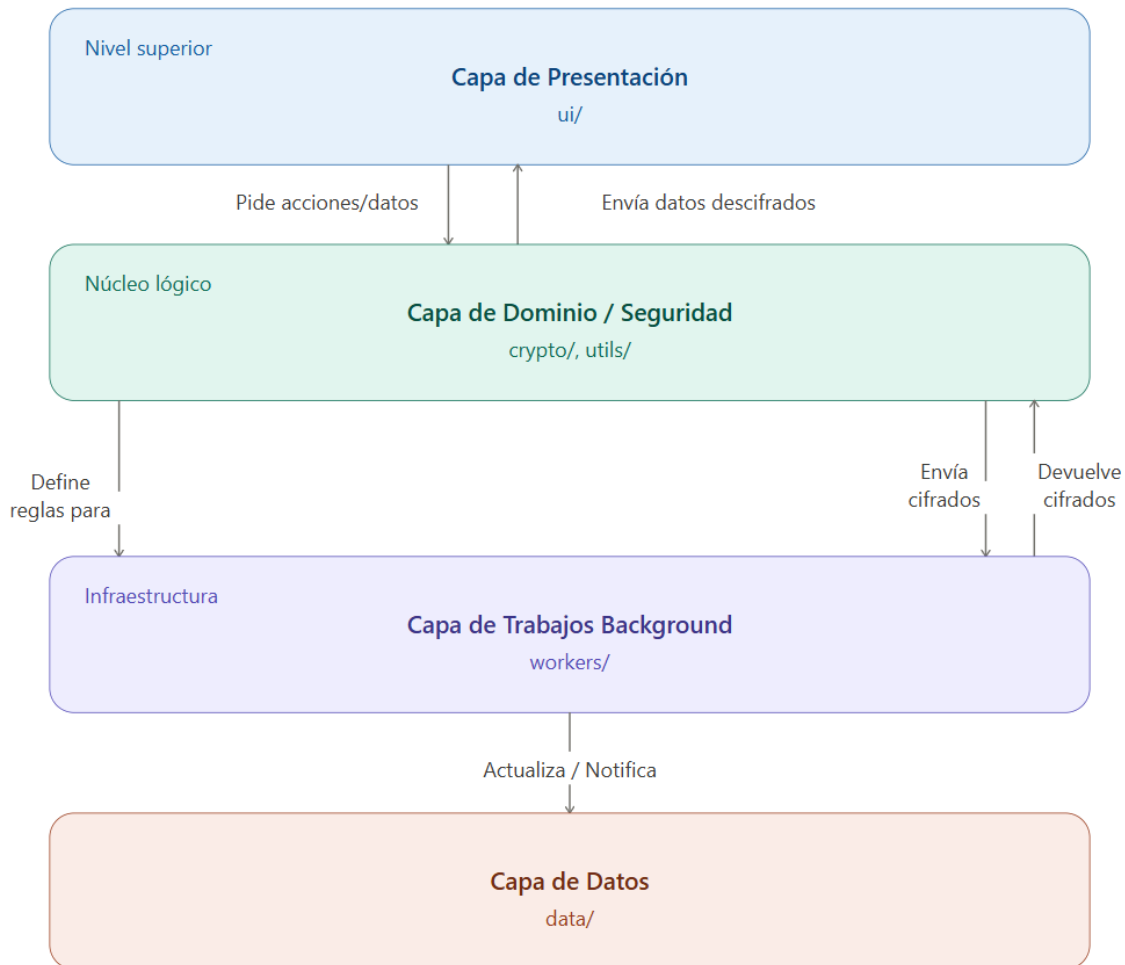
5. Diseño del sistema y arquitectura

5.1 Arquitectura general del sistema

Clef sigue una arquitectura cliente-servidor en la que el cliente (la aplicación Android) asume toda la responsabilidad de la seguridad criptográfica, y el servidor (Firebase) actúa únicamente como almacén opaco de datos cifrados.

Desde el punto de vista estructural, el código se organiza en cuatro capas bien diferenciadas:

- **Capa de presentación** (“ui”): Actividades y fragmentos que gestionan la interfaz de usuario. No contienen lógica de negocio ni operaciones criptográficas.
- **Capa de dominio / seguridad** (“crypto”, “utils”): Motor criptográfico, gestión de sesión, protección contra ataques, biometría y autofill. Es el núcleo de la aplicación.
- **Capa de datos** (“data”): Repositorios, gestión de Firebase y persistencia local. Recibe siempre datos ya cifrados de la capa superior.
- **Capa de trabajos en background** (“workers”): Tareas periódicas gestionadas por WorkManager, como las notificaciones de expiración.



5.2 Modelo de datos

El vault de Clef se estructura alrededor de dos entidades principales:

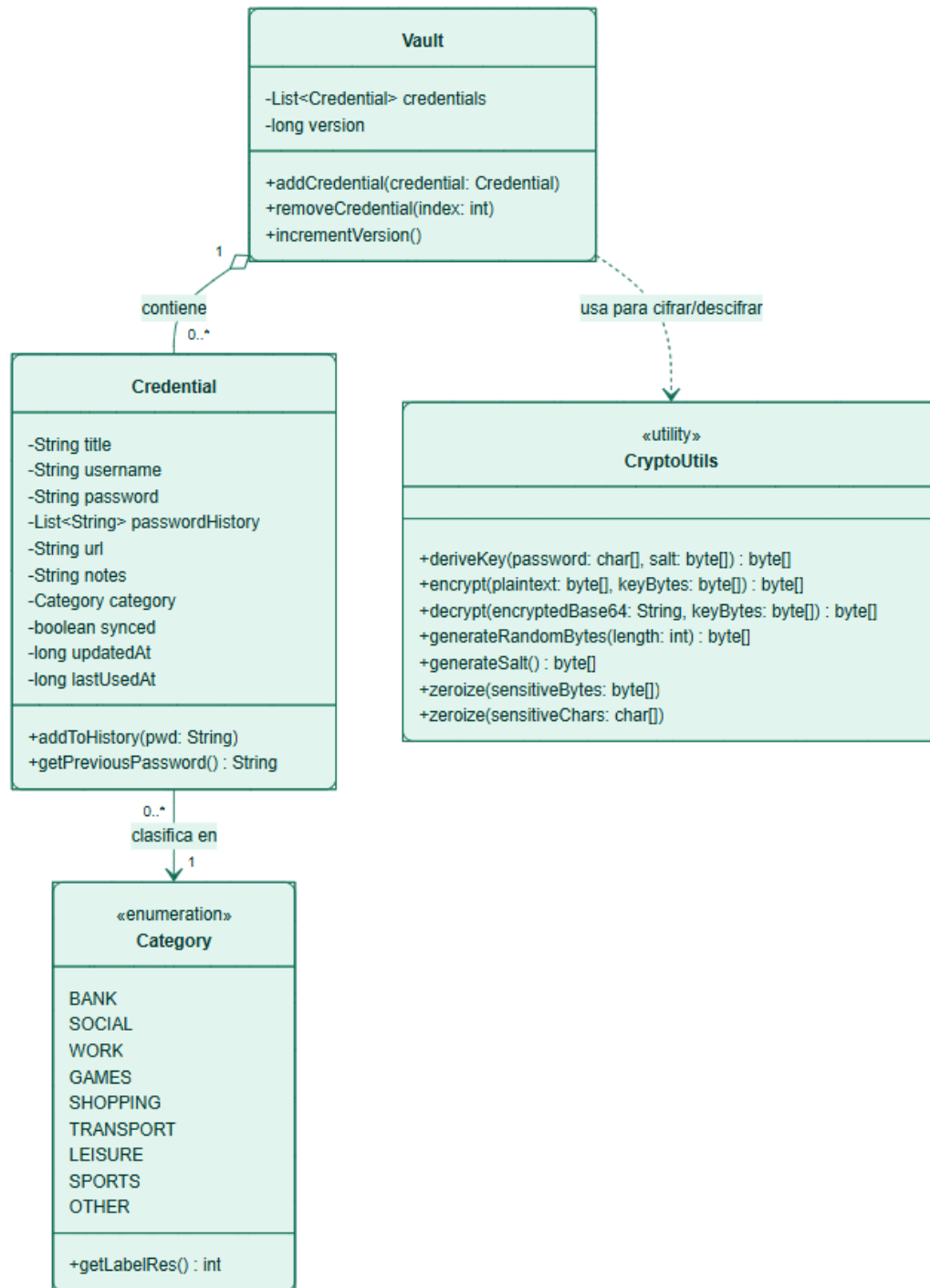
Vault: Es el contenedor principal. Agrupa todas las credenciales del usuario en una lista y mantiene un campo “version” que se incrementa con cada modificación. Este campo es esencial para la resolución de conflictos cuando hay cambios concurrentes entre el dispositivo local y la nube.

5. Diseño del sistema y arquitectura

Credential: Representa una cuenta guardada. Contiene el título del servicio, el nombre de usuario, la contraseña, la URL, notas adicionales, la categoría, una marca de tiempo de la última modificación, otra de la última vez que se usó para autofill, un historial de las últimas 5 contraseñas, la lista de apps asociadas aprendidas por autofill, y si está marcada como favorita.

La contraseña dentro de “Credential” se maneja como “char[]” en lugar de “String” en todos los puntos de la aplicación donde se trabaja con ella en claro. Esto es deliberado: los objetos “String” en Java son inmutables y permanecen en la memoria hasta que el recolector de basura los elimina, sin que el desarrollador pueda controlar cuándo ocurre. Un “char[]”, en cambio, puede sobrescribirse con ceros en el momento exacto en que deja de ser necesario, reduciendo la ventana temporal durante la cual la contraseña existe en memoria.

Las categorías disponibles están definidas como enumeración: “BANK”, “SOCIAL”, “WORK”, “GAMES”, “SHOPPING”, “TRANSPORT”, “LEISURE”, “SPORTS”, “OTHER”.

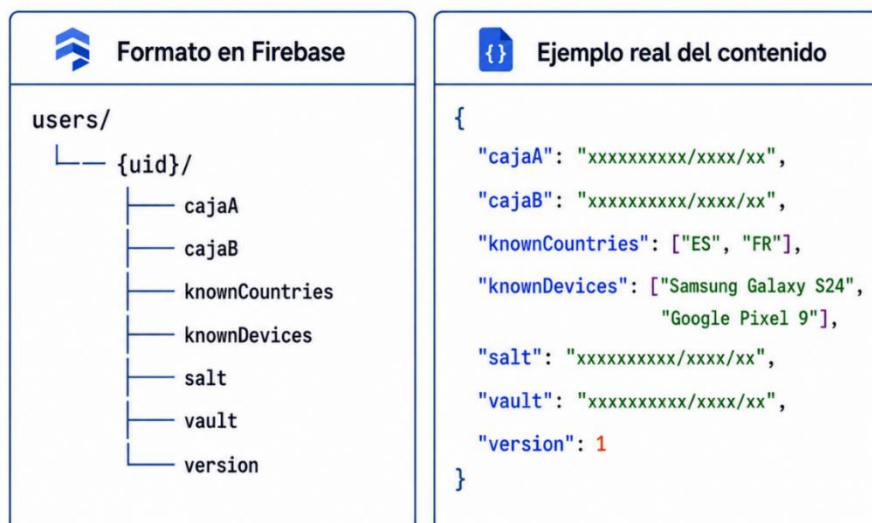


5. Diseño del sistema y arquitectura

En Firebase Firestore, cada usuario tiene un único documento bajo la colección “users/”, identificado por su UID de autenticación. Ese documento contiene:

- CajaA: La DEK cifrada con la KEK derivada de la contraseña maestra. Permite el login normal.
- CajaB: La misma DEK cifrada con la KEK derivada del PUK. Permite recuperar el acceso sin la contraseña maestra.
- KnownDevices: Array de strings con los modelos de dispositivo conocidos (formato FABRICANTE MODELO). Lo escribe checkLoginIp al detectar un dispositivo nuevo.
- KnownCountries: Array de códigos de país ISO 3166-1 alpha-2 conocidos. Lo obtiene checkLoginIp geocalizando la IP del login y lo guarda al detectar un país nuevo.
- Salt: 32 bytes aleatorios generados en el registro, en Base64. Usado para derivar las claves KEK-Master y KEK-PUK con PBKDF2.
- Vault: El vault completo (todas las credenciales serializado en JSON) cifrado con la DEK, en Base64.
- Version: Entero que se incrementa con cada escritura del vault. Lo usa la app para detectar y resolver conflictos de sincronización.

Estructura del usuario en Firebase



5.3 Arquitectura criptográfica: el modelo Zero-Knowledge

El principio fundamental de la arquitectura es que la clave que descifra los datos del usuario (la DEK, o *Data Encryption Key*) nunca se almacena en ningún sitio en abierto. Firebase recibe siempre una versión cifrada de esa clave, y el servidor no tiene forma de descifrarla porque la clave para hacerlo solo existe en la cabeza del usuario.

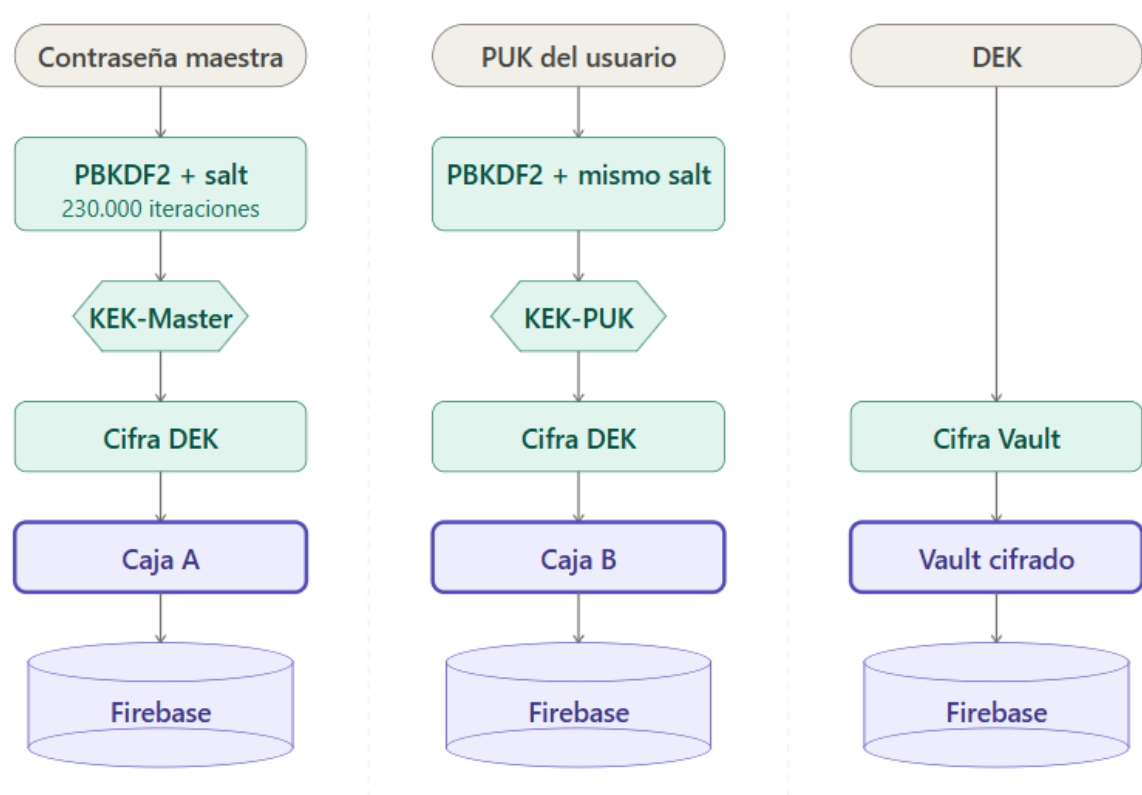
La arquitectura se construye sobre tres niveles:

Nivel 1 — Derivación de clave. Cuando el usuario introduce su contraseña maestra, la aplicación no la usa directamente para cifrar nada. En cambio, la combina con el “salt” almacenado en Firebase mediante PBKDF2WithHmacSHA256 con 230.000 iteraciones, obteniendo una KEK (*Key Encryption Key*) de 256 bits. El número de iteraciones está calibrado para que la operación tarde aproximadamente 400ms en un dispositivo moderno: suficiente para hacer inviable un ataque de diccionario masivo, pero imperceptible para el usuario en el desbloqueo normal.

Nivel 2 — Protección de la DEK. Durante el registro, se genera una DEK aleatoria de 256 bits que será la clave real de los datos. Esta DEK se cifra con la KEK derivada de la contraseña maestra, produciendo la *Caja A*, que se sube a Firebase. Paralelamente, se genera un PUK aleatorio de 16 bytes (presentado al usuario como una cadena hexadecimal de 32 caracteres), se deriva otra KEK a partir de él, y se cifra la misma DEK con esa segunda KEK, produciendo la *Caja B*. Así, tanto la contraseña maestra como el PUK son capaces de abrir la DEK, pero por caminos independientes.

5. Diseño del sistema y arquitectura

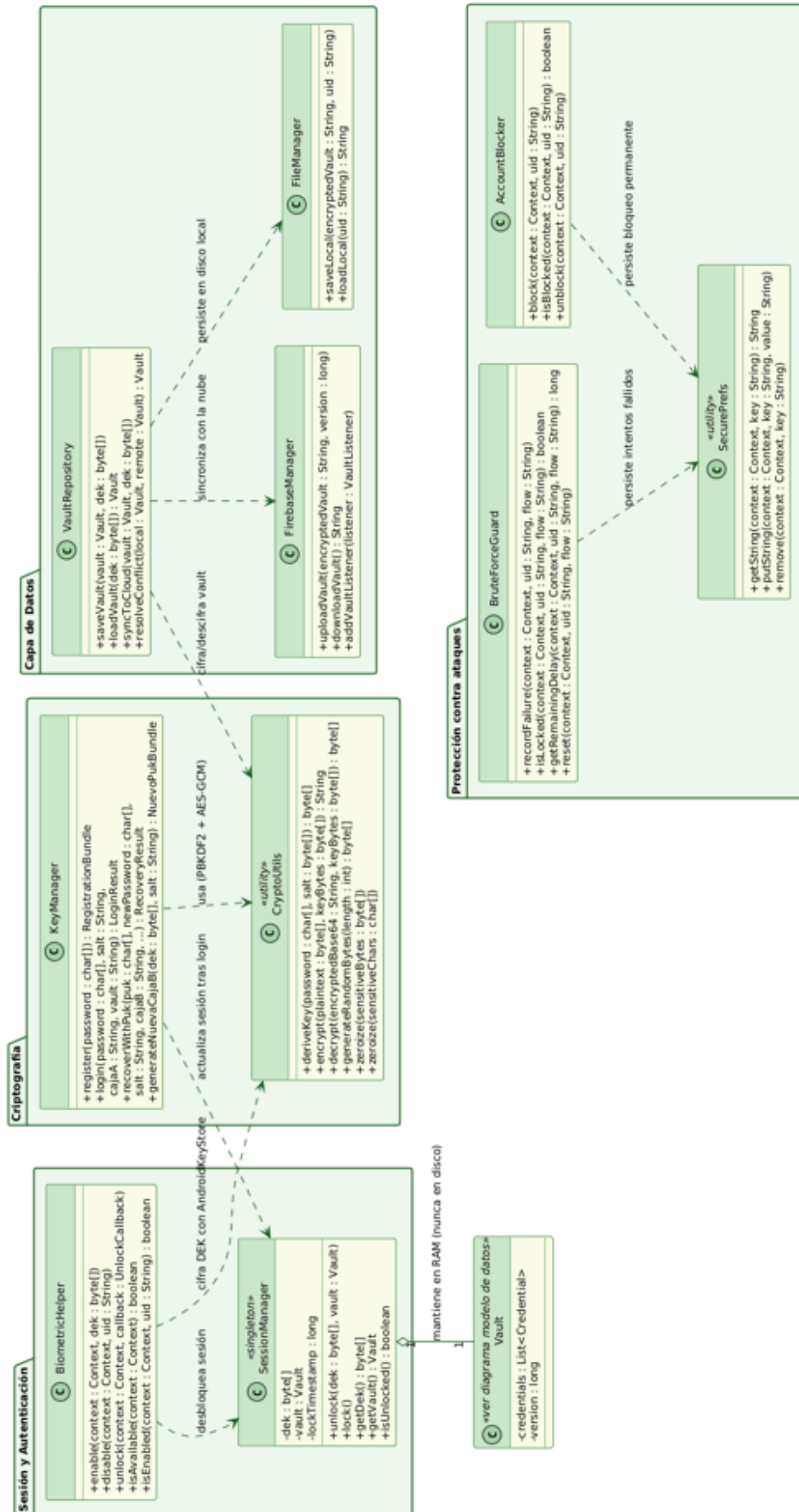
Nivel 3 — Cifrado del vault. El vault completo (todas las credenciales serializadas en JSON) se cifra con la DEK usando AES-256-GCM. El resultado es lo que se almacena en Firebase. Cada operación de cifrado usa un IV (vector de inicialización) de 12 bytes generado aleatoriamente, que va incluido en el texto cifrado resultante.



Lo que Firebase almacena en todo momento son únicamente Caja A, Caja B y el Vault cifrado. Sin la contraseña maestra o el PUK del usuario, estos tres elementos son datos completamente opacos.

Durante la sesión activa, la DEK se mantiene en el “SessionManager”, un singleton que vive en memoria RAM. Al bloquear la aplicación —ya sea manualmente, por timeout de inactividad, o por pasar al segundo plano— el “SessionManager” sobrescribe el array de bytes de la DEK con ceros antes de liberar la referencia.

5. Diseño del sistema y arquitectura



5.4 Flujos de autenticación

Registro:

Al crear una cuenta nueva, la aplicación ejecuta el proceso de registro en un hilo secundario para no bloquear la interfaz. Se genera el salt, la DEK, el PUK y ambas cajas mediante “KeyManager.register()”. A continuación, se sube todo a Firebase en una única escritura atómica. Si la escritura falla, no queda ningún dato parcial en el servidor. El PUK se muestra al usuario en la pantalla “ShowPukActivity” y se advierte explícitamente de que no podrá recuperarse. Una vez cerrada esa pantalla, el PUK desaparece de la memoria.

Desbloqueo con contraseña maestra:

“KeyManager.login()” deriva la KEK-Master a partir de la contraseña introducida y el salt descargado de Firebase, intenta descifrar la Caja A para obtener la DEK, y si tiene éxito, descifra el vault. Si la contraseña es incorrecta, AES-GCM detecta la manipulación y lanza “AEADBadTagException”, que la aplicación interpreta como contraseña incorrecta. “BruteForceGuard” registra el fallo y aplica el delay correspondiente.

Desbloqueo con biometría:

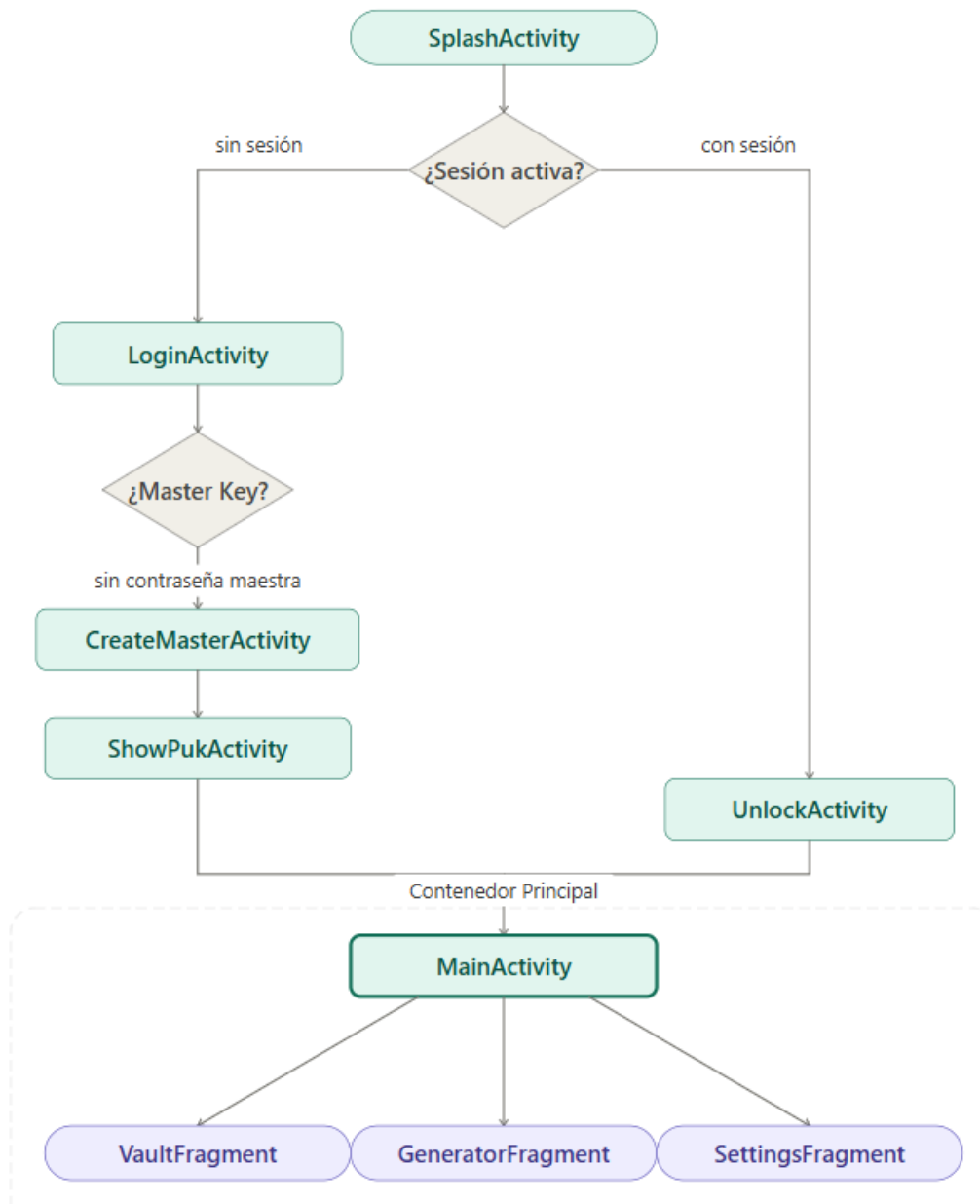
Al activar la biometría, la aplicación cifra la DEK con una clave generada en el Android Keystore y la almacena en “EncryptedSharedPreferences”. En el desbloqueo posterior, “BiometricHelper” muestra el diálogo de autenticación del sistema; si el usuario se autentica correctamente, el Keystore desbloquea la clave y se recupera la DEK. Esta opción no está disponible si han pasado más de 3 días desde la última verificación con contraseña maestra, como medida de seguridad adicional.

Recuperación con PUK:

El flujo de “RecoverVaultActivity” es análogo al del login normal, pero usando la Caja B en lugar de la Caja A. Si la derivación con el PUK introducido descifra correctamente la Caja B, se obtiene la DEK, y a partir de ella se genera una nueva Caja A con la nueva contraseña maestra y una nueva Caja B con un PUK recién generado, invalidando el anterior. El vault de datos no se toca: la DEK es la misma, por lo que todos los datos siguen siendo accesibles.

5.5 Estructura de navegación

El flujo de navegación de la aplicación sigue una estructura lineal en la fase de acceso, que desemboca en una navegación por pestañas en la pantalla principal:



La “MainActivity” utiliza un “BottomNavigationView” con tres pestañas: la bóveda de credenciales, el generador de contraseñas y los ajustes. El estado de cada fragmento se preserva al cambiar de pestaña.

5. Diseño del sistema y arquitectura

5.6 Diseño de interfaz

5:32

Crear cuenta

Regístrate en Clef

Correo electrónico

Contraseña

Confirmar contraseña

CREAR CUENTA

¿Ya tienes cuenta? [Iniciar sesión](#)

Creación de cuenta

Crear cuenta' and a footer with links 'TÉRMINOS · PRIVACIDAD · SOPORTE'."/>

5:31

Clef

Tu bóveda de contraseñas segura

Correo electrónico

Contraseña

¿Olvidaste tu contraseña?

ACCEDER

o continúa con

Continuar con Google

¿No tienes cuenta? [Crear cuenta](#)

TÉRMINOS · PRIVACIDAD · SOPORTE

Inicio de sesión

← Volver al login

Bienvenido de vuelta

Introduce tu contraseña maestra para acceder a tu bóveda

Contraseña maestra

Desbloquear

¿Olvidaste tu contraseña maestra? Usar código PUK

Clave maestra

5:35

Recuperar acceso

Introduce tu código PUK y elige una nueva Contraseña Maestra.

Código PUK (XXXX-XXXX-XXXX-...)

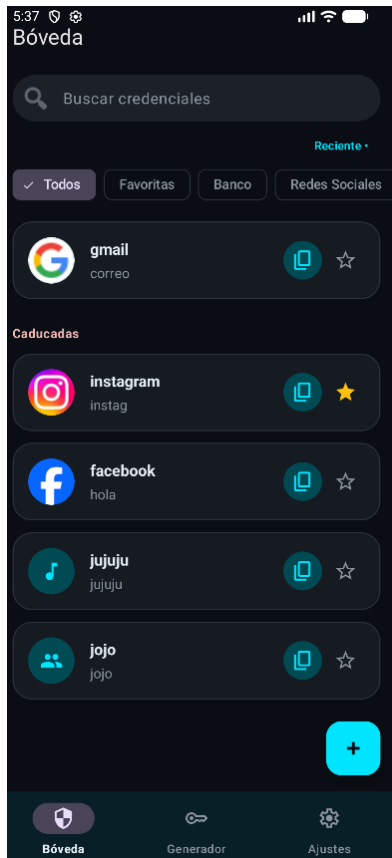
Nueva Contraseña Maestra

Confirmar nueva contraseña

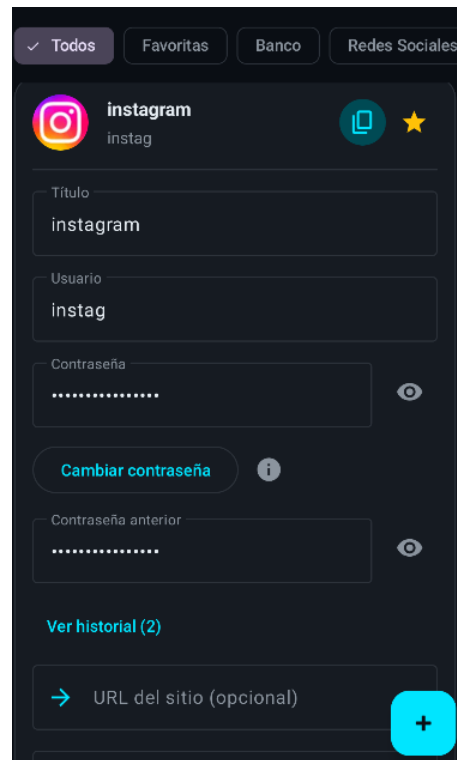
⚠ El código PUK solo puede usarse una vez. Tras la recuperación quedará invalidado.

Recuperar acceso

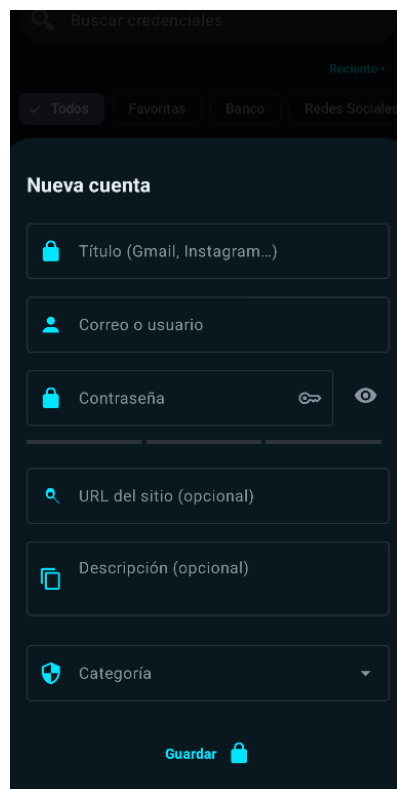
Recuperar clave maestra (PUK)



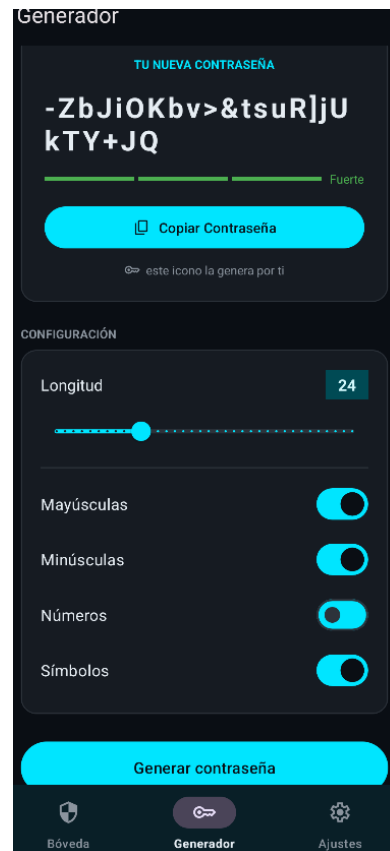
Bóveda



Ítem contraseña



Añadir contraseña



Generador de contraseñas

5. Diseño del sistema y arquitectura



Menú ajustes



Menú ayuda y soporte

La interfaz sigue Material Design 3, con una paleta de colores basada en verde oscuro que refuerza visualmente la identidad del proyecto. Los íconos de categoría son imágenes vectoriales que permiten identificar rápidamente el tipo de credencial en el listado. Las plataformas más comunes tienen su propio ícono; para el resto, se usa un ícono genérico basado en la inicial del título.

6. Desarrollo

6.1 Entorno de desarrollo

El desarrollo se realizó en Android Studio con el SDK de Android para Java 17. El control de versiones se gestionó con Git y el repositorio se alojó en GitHub. Para el diseño de diagramas se usaron PlantUML y draw.io. La configuración de Firebase se realizó desde la consola web de Firebase, vinculando el proyecto Android mediante el archivo “google-services.json”.

Las dependencias principales del proyecto se definen en el archivo “build.gradle” y se gestionan con Gradle. Se usó el BOM de Firebase (versión 34.10.0) para garantizar la compatibilidad entre los distintos módulos de Firebase. El “minSdk” se estableció en 28 (Android 9), que es el mínimo necesario para usar la API de Autofill en su versión completa y para tener acceso a “BiometricPrompt” con autenticadores fuertes.

6.2 Módulo de autenticación y gestión de sesión

La autenticación de usuarios se delega en Firebase Authentication, que soporta dos métodos: Google Sign-In mediante OAuth 2.0 y correo electrónico con contraseña. “AuthManager” encapsula toda la comunicación con Firebase Auth, mientras que la lógica de negocio del flujo (redirección tras login, verificación de si el usuario ya tiene contraseña maestra) reside en las actividades correspondientes.

Una vez autenticado el usuario en Firebase, la aplicación verifica en Firestore si ya tiene un documento de usuario (es decir, si ya ha creado una contraseña maestra).

Si no lo tiene, se redirige a “CreateMasterActivity”. En caso contrario, va directamente a “UnlockActivity”.

La “SessionManager” es el componente central de la gestión de sesión. Es un singleton que mantiene en memoria la DEK y el vault descifrado durante la sesión activa. Al bloquear:

La zeroización asegura que, aunque la memoria no se libere inmediatamente, el valor que queda en ese espacio de memoria es ceros, no la clave real.

El auto-lock se gestiona mediante un “Handler” que se activa cuando la aplicación pasa al segundo plano (“onStop” de “MainActivity”) y se cancela cuando vuelve al primer plano (“onStart”). El tiempo de espera es configurable por el usuario en los ajustes, con un valor por defecto de 5 minutos.

6.3 Motor criptográfico

El motor criptográfico se divide en dos clases: “CryptoUtils” y “KeyManager”.

“CryptoUtils” es una clase de utilidades estáticas que implementa las primitivas criptográficas:

- **Derivación de clave:** “PBKDF2WithHmacSHA256” con 230.000 iteraciones y clave de salida de 256 bits. El salt de 32 bytes garantiza que dos usuarios con la misma contraseña obtengan KEKs completamente diferentes.

- **Cifrado:** “AES/GCM/NoPadding” con clave de 256 bits e IV de 12 bytes generado aleatoriamente en cada operación. El resultado es “Base64([IV (12B)][ciphertext + tag GCM (16B)])”.

- **Descifrado:** Extrae el IV de los primeros 12 bytes, inicializa el cipher en modo DECRYPT, y verifica el tag de autenticación GCM. Si el tag no coincide (contraseña incorrecta, datos manipulados), lanza “AEADBadTagException”.

- **Zeroización:** Métodos estáticos para sobrescribir arrays de bytes y chars con ceros.

“KeyManager” usa “CryptoUtils” para orquestar los flujos completos de registro, login y recuperación, garantizando que todas las claves intermedias (KEK-Master, KEK-PUK) se zerorizan en bloques “finally” incluso si el proceso falla a mitad.

La elección de GCM sobre otros modos de AES (como CBC) fue deliberada: GCM proporciona cifrado autenticado, lo que significa que cualquier modificación del texto cifrado se detecta automáticamente en el descifrado. Esto elimina la necesidad de un HMAC separado y simplifica el código sin reducir la seguridad.

6.4 Gestión del vault y credenciales

El vault se gestiona a través de “VaultRepository”, que actúa como capa de abstracción entre la interfaz de usuario y las fuentes de datos (Firebase y almacenamiento local). Cualquier operación que modifique las credenciales pasa por el repositorio, que se encarga de cifrar el vault antes de persistirlo y de sincronizarlo con Firebase.

El “VaultFragment” muestra las credenciales mediante un “RecyclerView” con “VaultAdapter”. El listado admite búsqueda en tiempo real filtrando sobre la lista en

memoria, sin necesidad de consultar Firebase en cada pulsación de teclado. Las credenciales caducadas se muestran en una sección separada, visualmente diferenciada, al final del listado.

El “AddItemDialog” sirve tanto para crear como para editar credenciales. Integra directamente el generador de contraseñas para que el usuario pueda generar una contraseña nueva sin salir del diálogo. El historial de contraseñas se actualiza automáticamente al guardar un cambio: la contraseña anterior se añade al historial (máximo 5 entradas) y la nueva ocupa su lugar.

6.5 Sistema de Autofill

El sistema de Autofill es probablemente el módulo más complejo de la aplicación, porque requiere interactuar con el framework de Android Autofill de una forma que sea a la vez útil, segura y compatible con la variedad de aplicaciones existentes.

“ClefAutofillService” extiende “AutofillService” y responde a dos callbacks del sistema: “onFillRequest” (cuando se activa un formulario de login en otra app) y “onSaveRequest” (cuando se detecta que el usuario ha enviado un formulario con nuevas credenciales).

En “onFillRequest”, el servicio analiza el “AssistStructure” que proporciona el sistema, una representación del árbol de vistas de la pantalla actual. “FieldFinder” recorre este árbol buscando campos de tipo username y password, extrayendo sus “AutofillId” para poder rellenarlos después.

La respuesta que se devuelve al sistema varía según el estado de la sesión:

- **Si hay una sesión transitoria válida** (menos de 30 segundos desde la última autenticación biométrica en esta misma app): se incluye directamente el dataset con los valores reales, sin pedir autenticación adicional. Esto resuelve el problema de aplicaciones como Amazon o ciertas apps bancarias, que dividen el login en varias pantallas consecutivas y que de otro modo pedirían la huella dos o tres veces.

- **Si no hay sesión transitoria:** se devuelve un dataset que muestra "Desbloquear Clef" en el dropdown de sugerencias. Al seleccionarlo, se lanza "AutofillAuthActivity" mediante "PendingIntent".

"AutofillAuthActivity" pide autenticación biométrica y, si tiene éxito, descifra el vault y presenta las credenciales relevantes ordenadas por relevancia. El algoritmo de ranking puntúa cada credencial según tres criterios: si el "packageName" de la app objetivo coincide con los "packageHints" aprendidos (puntuación alta), si el dominio de la URL de la credencial coincide con el de la app (puntuación media) o si el título contiene el nombre de la app (puntuación baja). Si no hay ninguna coincidencia, se muestran todas las credenciales sin filtrar.

Al seleccionar una credencial, se rellena el formulario, se actualiza el campo "lastUsedAt" de la credencial y se registra el "packageName" en "packageHints" para mejorar el ranking en usos futuros.

6.6 Sincronización y resolución de conflictos

La sincronización con Firebase usa un modelo de **optimistic locking**: cada vault tiene un campo “version” que se incrementa con cada escritura. Al subir cambios, la aplicación ejecuta una transacción en Firestore que verifica que la versión remota coincide con la esperada antes de escribir. Si no coincide (porque otro dispositivo modificó el vault mientras tanto), la transacción falla y se descarga la versión remota.

La resolución de conflictos es conservadora: se mantienen las credenciales marcadas como sincronizadas del servidor remoto, y se añaden encima las credenciales locales que aún no habían sido sincronizadas. Esto garantiza que no se pierda ningún dato, aunque en casos extremos podría crear duplicados que el usuario tendría que limpiar manualmente.

Además de la sincronización manual, “FirebaseManager” registra un listener de Firestore que recibe notificaciones en tiempo real cuando el vault remoto cambia. Esto permite que los cambios realizados desde otro dispositivo aparezcan automáticamente en la pantalla, sin necesidad de recargar.

6.7 Notificaciones y expiración de contraseñas

“ExpiryHelper” calcula el estado de expiración de cada credencial en función de su “updatedAt” y del período configurado por el usuario. Hay cuatro estados posibles:

- **“NONE”**: La credencial nunca se ha modificado.
- **“OK”**: La contraseña tiene menos del 80% del período de validez consumido.
- **“WARNING”**: Entre el 80% y el 100%.
- **“EXPIRED”**: Ha superado el período configurado.

Para las notificaciones, se usa “WorkManager” con “PasswordExpiryWorker”, que se ejecuta una vez al día. Este worker lee únicamente los metadatos de las credenciales (título y “updatedAt”, almacenados en “SecurePrefs” sin incluir contraseñas) para calcular cuántas están en estado “WARNING” o “EXPIRED”, y envía una notificación si procede. La separación entre los metadatos de expiración y el vault cifrado es intencionada: el worker puede hacer su trabajo sin necesitar acceso a la DEK, que podría no estar disponible si la sesión está bloqueada.

6.8 Sistemas de correos al usuario

1. Verificación de email — enviado por Firebase Authentication al registrarse con email y contraseña. El usuario debe hacer clic en el enlace para activar la cuenta. Sin verificar, no puede acceder a la app.

2. Recuperación de contraseña de Firebase — enviado por Firebase Authentication cuando el usuario solicita restablecer su contraseña de acceso a Firebase (no la contraseña maestra). Se genera desde la pantalla de login.

3. Alerta de acceso sospechoso — enviado por la Cloud Function checkLoginIp cuando se detecta un login desde un dispositivo o país no reconocido. Incluye el modelo del dispositivo, la ubicación (ciudad, región, país) y la fecha y hora del acceso.

4. Notificación de cambio de contraseña maestra — enviado por la Cloud Function notifyPasswordChange cuando el usuario cambia su contraseña maestra. Incluye la fecha y hora del cambio. Sirve como aviso de seguridad por si el cambio no fue iniciado por el usuario legítimo.

Los correos 1 y 2 los gestiona Firebase directamente con sus plantillas. Los correos 3 y 4 los envía la app desde security.clef@gmail.com usando Nodemailer con una cuenta de Gmail dedicada.

7. Seguridad

7.1 Modelo de cifrado

El modelo de seguridad de Clef se basa en el principio de que un sistema es tan seguro como su punto más débil, y que ese punto más débil debe estar bajo el control exclusivo del usuario. Todo el cifrado usa AES-256-GCM, que combina confidencialidad e integridad en una sola operación. La derivación de contraseñas usa PBKDF2WithHmacSHA256, que es el algoritmo recomendado por NIST SP 800-132 para este propósito.

El salt de 32 bytes garantiza que ataques de tabla arcoíris son inviables: incluso si dos usuarios usaran la misma contraseña, sus KEKs serían completamente diferentes. Las 230.000 iteraciones de PBKDF2 hacen que derivar una clave lleve aproximadamente 400ms en hardware moderno. Un atacante que quisiera probar un millón de contraseñas tardaría más de 4 días con un solo núcleo, y la escalabilidad de un ataque GPU queda limitada por la naturaleza secuencial de PBKDF2.

7.2 Protecciones implementadas

BruteForceGuard protege los flujos de desbloqueo y recuperación con un sistema de delays progresivos: tras el primer fallo, el usuario debe esperar 10 segundos para volver a intentarlo; tras el segundo, 30; luego 60, 120 y 240. Los contadores se almacenan en EncryptedSharedPreferences, por lo que persisten aunque la aplicación se cierre y se vuelva a abrir, impidiendo que reiniciar la app sea una forma de saltarse el bloqueo. En dispositivos sin TEE disponible, los contadores se mantienen en memoria durante la sesión activa.

AccountBlocker entra en acción si el usuario agota los 5 intentos disponibles tanto en el desbloqueo con contraseña como en la recuperación con PUK. En ese caso, la cuenta se bloquea permanentemente y solo puede desbloquearse con intervención del equipo de soporte técnico. Esto es una medida extrema, pero coherente con una aplicación cuyo modelo de seguridad no puede permitir intentos ilimitados.

Android Keystore aísla las claves biométricas en el TEE (Trusted Execution Environment) del dispositivo, un procesador separado del procesador principal que no puede ser accedido ni siquiera por el sistema operativo. Las claves generadas ahí no pueden exportarse, y están configuradas para invalidarse si el usuario añade o elimina credenciales biométricas del dispositivo.

Root detection. La aplicación verifica en el arranque si el dispositivo tiene root. Un dispositivo con root puede tener comprometida la separación entre procesos que garantiza la privacidad de los datos de la app, por lo que la aplicación muestra un diálogo informativo y se cierra, impidiendo su uso en ese entorno.

Flag FLAG_SECURE. El flag se aplica globalmente a todas las pantallas de la aplicación mediante un `ActivityLifecycleCallbacks` registrado en la clase `Application`. Esto impide que cualquier pantalla aparezca en capturas de pantalla y en la vista previa del selector de apps, sin necesidad de configurarlo pantalla por pantalla

7.3 Vectores de ataque y mitigaciones

Vector de ataque	Mitigación implementada
Acceso físico al dispositivo desbloqueado	Auto-lock por inactividad configurable
Ataque de fuerza bruta a la contraseña maestra	BruteForceGuard + AccountBlocker
Robo de la base de datos Firebase	Zero-knowledge: todos los datos están cifrados con AES-256-GCM

Keylogger capturando la contraseña maestra	Uso de “char[]” con zeroización inmediata
Volcado de memoria RAM	DEK zerizada al bloquear “FLAG_SECURE” en pantallas sensibles
Autofill de phishing (app maliciosa suplantando a otra)	Ranking por “packageHints” aprendidos; confirmación biométrica siempre requerida
Contraseñas débiles del usuario	Indicador de fortaleza en tiempo real; generador seguro integrado
Acceso a datos tras cambio biométrico	Keystore invalida claves biométricas si cambian las credenciales del dispositivo

8. Software y herramientas utilizadas

Herramienta/Tecnología	Versión	Uso en el proyecto
Android Studio	-	Entorno de desarrollo integrado
Java	17	Lenguaje de programación principal
Gradle	8+	Sistema de compilación y gestión de dependencias
Firebase Authentication	BOM 34.10.0	Autenticación de usuarios (Google + email)

Firestore	BOM 34.10.0	Base de datos en la nube
Functions	BOM 34.10.0	Control de la BBDD, comunicación con usuario mediante email y detección de dispositivo e IP
Console	-	Gestión de la infraestructura en la nube
androidx.security (EncryptedSharedPreferences)	1.1.0-alpha06	Almacenamiento cifrado de preferencias
androidx.biometric (BiometricPrompt)	1.1.0	Autenticación biométrica
androidx.autofill	1.3.0	Integración con el framework de Autofill de Android
androidx.work (WorkManager)	2.9.0	Tareas periódicas en background (notificaciones)
GSON	2.11.0	Serialización/deserialización del vault a JSON
Lottie	6.4.1	Animaciones en la interfaz
Glide	4.16.0	Carga de imágenes e íconos
Material Design	3	Componentes de interfaz de usuario
Git	-	Control de versiones
GitHub	-	Alojamiento del repositorio
PlantUML	-	Generación de diagramas UML
draw.io (diagrams.net)	-	Diagramas de arquitectura

9. Pruebas y validación

9.1 Estrategia de pruebas

Dado el carácter de seguridad de la aplicación, las pruebas se centraron principalmente en dos áreas: la corrección funcional de los flujos de usuario (registro, login, recuperación) y la validación de que los mecanismos de protección se comportan como se espera bajo condiciones adversas.

No se desarrollaron pruebas unitarias automatizadas para el motor criptográfico, dado que las primitivas que se usan (AES-GCM, PBKDF2) son parte de la biblioteca estándar de Java y están validadas de forma independiente. Las pruebas se realizaron de forma manual sobre un dispositivo físico y en el emulador de Android Studio.

9.2 Casos de prueba funcionales

ID	Caso de prueba	Resultado esperado	Resultado
CP-01	Registro con Google en dispositivo nuevo	Cuenta creada, PUK mostrado, acceso al vault	OK
CP-02	Login con contraseña maestra correcta	Vault descifrado, acceso concedido	OK
CP-03	Login con contraseña maestra incorrecta	Mensaje de error, delay aplicado	OK
CP-04	5 intentos fallidos de login	Cuenta bloqueada, mensaje informativo	OK
CP-05	Recuperación con PUK correcto	Vault descifrado, acceso concedido	OK

CP-06	Recuperación con PUK correcto	Nueva contraseña establecida, nuevo PUK generado	OK
CP-07	Recuperación con PUK incorrecto	Error, delay aplicado	OK
CP-08	Añadir credencial nueva	Credencial guardada en vault, sincronizada con Firebase	OK
CP-09	Editar contraseña de una credencial	Historial actualizado, cambio persistido	OK
CP-10	Generar contraseña con todos los tipos activos	Contraseña contiene al menos un carácter de cada tipo	OK
CP-11	Autofill en app con credencial asociada	Credencial correcta aparece en primer lugar	OK
CP-12	Autofill en flujo multi-pantalla (dos pasos)	Una sola autenticación biométrica para los dos pasos	OK
CP-13	Auto-lock tras inactividad	Vault bloqueado, pantalla de desbloqueo mostrada	OK
CP-14	Modificación en otro dispositivo	Cambio aparece automáticamente en el dispositivo	OK
CP-15	Exportar e importar vault	Vault restaurado correctamente tras importación	OK
CP-16	Contraseña caducada	Notificación recibida al día siguiente, credencial en sección "Caducadas"	OK

9.3 Pruebas de seguridad

ID	Caso de prueba	Descripción	Resultado
CS-01	Verificación de datos en Firebase	Consultar directamente Firestore y comprobar que ningún valor es legible	OK
CS-02	DEK no persiste en disco	Inspeccionar el almacenamiento local tras bloquear la app	OK
CS-03	Zeroización de DEK al bloquear	Inspeccionar el almacenamiento local tras bloquear la	OK
CS-04	BruteForceGuard persiste tras reinicio	Intentar 3 fallos, cerrar la app, reabrir y verificar el delay	OK
CS-05	Captura de pantalla bloqueada	Intentar captura en pantalla de credencial	OK

9.4 Pruebas Unitarias y Validación del Sistema

Para garantizar la estabilidad y seguridad de Clef, se ha implementado una suite de pruebas unitarias enfocadas en los componentes más críticos de la aplicación: el motor criptográfico, la gestión de claves y la validación de credenciales. A continuación, se detalla la cobertura de la suite de pruebas mediante las clases evaluadas:

Clase de Prueba	Clase de Prueba	Escenarios Clave Evaluados	Resultado
Password StrengthHelperTest	Verificar el algoritmo de evaluación de fortaleza de las contraseñas maestras y generadas.	- Manejo de casos nulos, vacíos y espacios en blanco. Clasificación correcta de contraseñas Débiles, Medias y Fuertes según entropía.- Resiliencia ante caracteres especiales (Unicode).	OK
CryptoUtilsTest	Garantizar la integridad de las primitivas criptográficas de bajo nivel (AES-256-GCM y PBKDF2).	- El cifrado del mismo dato genera <i>ciphertexts</i> distintos gracias a la aleatoriedad del vector de inicialización (IV).- Lanzamiento de excepciones ante intentos de descifrado con claves incorrectas.- Detección de manipulación de datos en el servidor (AEADBadTagException)	OK

KeyManagerTest	Validar la arquitectura lógica Zero-Knowledge y el correcto flujo de las estructuras de datos.	-Flujo de Registro (generación de la Caja A, Caja B y el PUK). -Flujo de Login y reconstrucción en memoria de la Bóveda. -Flujo de Recuperación mediante PUK, con invalidación criptográfica y generación de una nueva Caja B.	OK
Password Generator Test	Validar que el motor criptográfico de generación de claves (SecureRandom) cumple con los requisitos de entropía	- Respeto estricto de las longitudes mínimas y máximas solicitadas. - Garantía de inclusión de los tipos de caracteres seleccionados (mayúsculas, símbolos, etc.). -Verificación de aleatoriedad real en generaciones consecutivas.	OK

10. Despliegue

10.1 Requisitos previos

Para compilar y ejecutar Clef en un dispositivo de desarrollo, son necesarios:

- Android Studio Ladybug o superior
- JDK 17
- Android SDK con API 28 o superior instalada
- Un proyecto de Firebase configurado con Authentication (Google + email), Firestore y Functions habilitados (plan Blaze obligatorio para Functions)
- El archivo google-services.json del proyecto Firebase, ubicado en app/
- Las Cloud Functions desplegadas en el proyecto de Firebase, con el secreto GMAIL_APP_PASSWORD configurado en Google Cloud Secret Manager
- Un dispositivo físico o emulador con Android 9 (API 28) o superior

10.2 Configuración de Firebase

El proyecto requiere que en la consola de Firebase estén habilitados:

- **Firebase Authentication** con los proveedores Google y Email/contraseña
- **Cloud Firestore** con las reglas de seguridad configuradas para que cada usuario solo pueda leer y escribir su propio documento
- **Firebase Functions (plan Blaze)** con las funciones checkLoginIp, notifyPasswordChange y deleteAccountHttp desplegadas, y el secreto GMAIL_APP_PASSWORD configurado en Google Cloud Secret Manager

- **El fingerprint SHA-1** del certificado de firma registrado en la configuración del proyecto de Firebase (necesario para Google Sign-In)

10.3 Compilación y ejecución

Para compilar y generar el APK firmado en modo release, es necesario tener Android Studio instalado (versión Hedgehog o superior recomendada) junto con el SDK de Android para API nivel 28-36.

Pasos para compilar:

1. Clonar el repositorio y abrir el proyecto en Android Studio.
2. Colocar el archivo google-services.json (descargado desde la consola de Firebase del proyecto) en la carpeta app/.
3. En el menú superior, seleccionar Build → Generate Signed Bundle / APK.
4. Elegir APK, configurar o crear el keystore de firma, y seleccionar el variant release.
5. Android Studio invocará Gradle, que ejecutará R8/ProGuard con las reglas definidas en proguard-rules.pro para ofuscar el código y reducir el tamaño del binario.
6. El APK resultante se generará en app/build/outputs/apk/release/app-release.apk.

Para instalar en un dispositivo físico o emulador:

adb install app/build/outputs/apk/release/app-release.apk

También es posible ejecutar directamente en modo debug desde Android Studio pulsando Run lo que desplegará una build debug sin ofuscación en el dispositivo o emulador conectado. La build debug incluye logs adicionales y no activa todas las protecciones de seguridad (por ejemplo, FLAG_SECURE sigue activo pero los logs de depuración del motor criptográfico son más verbosos).

Nota: La aplicación requiere un dispositivo o emulador con Android 9 (API 28) como mínimo. El servicio de Autofill y la autenticación biométrica fuerte no están disponibles en emuladores estándar; para probarlos es necesario un dispositivo físico.

11. Memoria económica

11.1 Costes de hardware

Concepto	Coste total	Amortización mensual	Meses de uso	Coste imputable
Ordenador portátil (x3)	1800 €	37,5 €	3,5	131,25 €
Dispositivo Android de pruebas	300 €	12,5 €	3,5	43,75 €
Total hardware				175,00 €

La amortización de los portátiles se calcula sobre una vida útil de 4 años (48 meses). La del dispositivo Android sobre 2 años (24 meses). Se imputa únicamente el período efectivo de uso del proyecto (3,5 meses).

11.2 Costes de software y licencias

Herramienta	Licencia	Coste
Android Studio	Gratuito (Apache 2.0)	0 €
Java 17 (OpenJDK)	Gratuito	0 €
Firebase (Plan Blaze)	Pago por uso	0 €
Github	Gratuito (Plan Free)	0 €
PlantUML/draw.io	Gratuito	0 €
Node.js	Gratuito (MIT)	0 €
Firebase CLI	Gratuito (Apache 2.0)	0 €
Ip-api.com	Gratuito (Plan Free)	0 €
Total Software		0 €

Todas las herramientas utilizadas son de uso gratuito. El plan Blaze de Firebase cubre ampliamente el volumen de uso de un proyecto académico.

11.3 Costes de desarrollo (recursos humanos)

Concepto	Horas/persona	Personas	Total horas	Tarifa	Coste
Análisis y diseño	25 h	3	75 h	10 €/h	750 €
Desarrollo	110 h	3	330 h	10 €/h	
Pruebas	20 h	3	60 h	10 €/h	
Documentación	20 h	3	60 h	10 €/h	
Total RRHH	175 h	3	525 h		5250 €

La tarifa de 10 €/h corresponde al coste estimado de un desarrollador Android junior en España (rango 18.000–24.000 € brutos anuales). El equipo está formado por 3 desarrolladores con dedicación equivalente a lo largo de las 14 semanas del proyecto.

11.4 Coste total del proyecto

Concepto	Coste
Hardware	175,00 €
Software	0 €
Recurso humanos	5250,00 €
TOTAL	5425,00 €

12. Conclusiones y mejoras futuras

12.1 Conclusiones

Clef es el proyecto más ambicioso que hemos abordado en el ciclo. Mezcla conceptos de criptografía aplicada, integración con APIs del sistema operativo, sincronización en tiempo real y diseño de interfaz, y todos esos elementos tienen que funcionar correctamente juntos para que la experiencia sea segura y usable al mismo tiempo.

Lo que más nos ha aportado es el trabajo con el motor criptográfico. Entender de verdad por qué se usan 230.000 iteraciones en PBKDF2, por qué GCM es mejor que CBC para este caso de uso, por qué la DEK debe existir solo en RAM y zerorizarse al bloquear... son decisiones que no están en ningún tutorial, sino en las especificaciones y en los papers de seguridad. Eso nos ha obligado a ir más allá del ciclo y buscar fuentes más técnicas.

El sistema de Autofill también fue un reto importante. Android pone muchas restricciones sobre lo que un servicio de autofill puede hacer y cuándo, y adaptarse a la variedad de aplicaciones con flujos de login distintos requirió varias iteraciones hasta llegar a la sesión transitoria como solución.

El resultado es una aplicación que, en términos de arquitectura de seguridad, está a la altura de soluciones comerciales. No en escala ni en recursos, pero sí en los principios que la gobiernan: zero-knowledge real, cifrado en cliente, claves que nunca tocan el servidor.

12.2 Dificultades encontradas

El portapapeles como superficie de ataque

Una de las primeras dificultades encontradas con la seguridad real apareció al implementar la copia de contraseñas al portapapeles. El portapapeles de Android es accesible a cualquier aplicación con la API adecuada, lo que convierte el gesto más intuitivo del usuario copiar y pegar su contraseña, siendo una pequeña brecha de exposición. La solución obvia era limpiar el portapapeles tras un intervalo corto, pero surgieron dos problemas en cascada: en Android 10 y superior la API `clearPrimaryClip()` existe pero su comportamiento varía entre fabricantes; y en Android 13+ el sistema muestra al usuario una notificación cada vez que una app lee el portapapeles, lo que hacía que el flujo pareciera intrusivo. Implementamos `ClipboardHelper` con limpieza automática a los 45 segundos para contraseñas normales y 5 minutos para el PUK, marcando el clip como sensible en Android 13+ **mediante** `EXTRA_IS_SENSITIVE` para que el sistema lo trate con discreción. Sin embargo, la comprobación de que el portapapeles es una superficie de ataque estructural no completamente solucionable desde la app fue lo que terminó de motivar la implementación del sistema de Autofill: si el usuario puede rellenar el campo de contraseña directamente desde Clef sin pasar por el portapapeles, el riesgo desaparece por completo.

La sincronización con Firebase y la resolución de conflictos

La sincronización del vault con Firestore resultó ser considerablemente más compleja de lo esperado. El primer diseño usaba una escritura simple: guardar el vault cifrado y ya. El problema apareció en cuanto consideramos el caso de uso más básico: el usuario tiene la app en el móvil y en una tablet, edita una credencial en cada dispositivo sin conexión, y luego ambos intentan sincronizar. Sin control de versiones, la última escritura gana y los cambios del otro dispositivo desaparecen silenciosamente.

12. Conclusiones y mejoras futuras

La solución implementada combina un campo **version** en el documento de Firestore con transacciones atómicas (**runTransaction**). Antes de escribir, se comprueba que la versión remota coincide con la que tenía el cliente al descargar; si no coincide, se detecta el conflicto y se ejecuta una fusión: se descarga el vault remoto, se integran las credenciales locales no sincronizadas, y se reintenta la escritura. Esto requirió rediseñar la arquitectura de **VaultRepository** varias veces y gestionar correctamente el ciclo de vida de la DEK en todos los caminos del código, incluyendo los errores.

La integración del servicio de Autofill de Android

El servicio de Autofill de Android tiene una curva de aprendizaje elevada y una documentación oficial que asume casos de uso más simples que el nuestro. Las principales dificultades fueron tres: la detección fiable de campos de login en aplicaciones que no declaran **autofillHints** explícitos (la mayoría), la gestión de flujos de autenticación multi-pantalla como los de Amazon u OpenAI donde el email y la contraseña aparecen en pantallas distintas, y la compatibilidad con las inline suggestions del teclado en Android 11+.

Para el primer problema desarrollamos **FieldFinder**, un analizador heurístico del **AssistStructure** que combina cuatro señales de detección: hints declarados, **InputType**, atributos del nodo y el texto del label precedente. Para el segundo, implementamos **AutofillTransientSession**, una sesión en memoria de 30 segundos que evita pedir la biometría dos veces en el mismo flujo. Para el tercero, añadimos soporte de **InlinePresentation** condicionado a API 30+ sin romper la compatibilidad hacia atrás.

12.3 Mejoras futuras

- **Versión iOS / multiplataforma:** La arquitectura criptográfica es independiente del sistema operativo y podría reimplementarse en Swift o en un framework multiplataforma como Kotlin Multiplatform.

- **Autoauditoría de contraseñas:** Integración con la API de Have I Been Pwned para avisar al usuario si alguna de sus contraseñas aparece en bases de datos de brechas conocidas, usando el protocolo k-anonymity para no revelar las contraseñas al servicio externo.
- **Compartir credenciales entre usuarios:** Implementar un sistema de compartición cifrada de extremo a extremo, similar al de Bitwarden Organizations, usando criptografía asimétrica (RSA o curvas elípticas).
- **Soporte para passkeys (FIDO2):** El estándar de passkeys está desplazando a las contraseñas tradicionales en las principales plataformas. Añadir soporte para almacenar y gestionar passkeys sería una evolución natural de la app.
- **Aplicación de escritorio:** Una versión de escritorio (Windows/macOS/Linux) con sincronización con el mismo vault, usando la misma arquitectura criptográfica.
- **Auditoría de seguridad externa:** Someter el código criptográfico a una revisión por parte de expertos externos para validar la implementación.

12.4 Escalabilidad de Soporte y Ayuda al Usuario

En el ciclo de vida de una aplicación de gestión de credenciales, el mayor cuello de botella operativo al escalar la base de usuarios es la gestión de incidencias de soporte, particularmente las relacionadas con la pérdida de acceso. Para garantizar que Clef pueda escalar de forma masiva sin un incremento lineal en los costes de atención al cliente, se ha diseñado una estrategia de *Zero-Touch Support* (Soporte de Intervención Cero) basada en la autogestión y la prevención proactiva:

- **Eliminación del Cuello de Botella Principal (Zero-Knowledge):** En los sistemas tradicionales, las peticiones de restablecimiento de contraseña escalan de forma lineal con el número de usuarios. Debido a la arquitectura *Zero-Knowledge* de Clef, el equipo de administración carece de capacidad técnica para resetear contraseñas maestras. La recuperación recae exclusivamente en un proceso criptográfico automatizado mediante el código PUK (*Personal Unblocking Key*). Esto convierte el coste operativo del problema de soporte más común en $O(1)$, reduciendo a cero la carga del servidor y del personal humano.

13. Bibliografía y fuentes consultadas

- **Soporte de Nivel 1 Descentralizado (HelpSupportSheet):** Para absorber la curva de aprendizaje de una base de usuarios en constante crecimiento sin requerir un equipo de *Help Desk*, Clef integra un componente de soporte nativo en la interfaz. Este módulo actúa como una capa de Nivel 1 automatizada, resolviendo dudas de funcionamiento en el propio dispositivo y educando al usuario sobre la importancia crítica de conservar su PUK de recuperación.
- **Portabilidad y Cumplimiento Normativo Autónomo (ImportExportDialog):** Las solicitudes de extracción de datos, necesarias para cumplir con normativas de privacidad (como el RGPD), suelen requerir intervención administrativa. Clef delega esta carga operativa en el cliente. El usuario tiene autonomía total para extraer, importar o migrar sus credenciales en texto plano cuando lo desee, previniendo el *vendor lock-in* y eliminando la generación de tickets de soporte por retención de datos.
- **Prevención Proactiva de Incidencias (PasswordExpiryWorker):** El soporte reactivo tras una vulneración de cuentas genera una alta carga de asistencia. Clef mitiga este riesgo en su origen mediante trabajos en segundo plano (*WorkManager*) que monitorizan la salud y caducidad de las contraseñas de forma estrictamente local. Al alertar al usuario para que renueve sus claves de forma proactiva, el sistema reduce drásticamente la creación de incidencias de soporte derivadas de ataques externos por el uso de contraseñas obsoletas.

13. Bibliografía y fuentes consultadas

Estándares y documentación técnica

- NIST SP 800-132 — Recommendation for Password-Based Key Derivation: <https://csrc.nist.gov/publications/detail/sp/800-132/final>
- NIST SP 800-38D — Recommendation for Block Cipher Modes of Operation: GCM and GMAC: <https://csrc.nist.gov/publications/detail/sp/800-38d/final>

- NIST FIPS 197 — Advanced Encryption Standard (AES):
<https://doi.org/10.6028/NIST.FIPS.197>
- Android Developers — Security best practices:
<https://developer.android.com/topic/security/best-practices>
- Android Developers — Android Keystore System:
<https://developer.android.com/privacy-and-security/keystore>
- Android Developers — Cryptography:
<https://developer.android.com/guide/topics/security/cryptography>
- Android Developers — AutofillService API:
<https://developer.android.com/reference/android/service/autofill/AutofillService>
- Android Developers — BiometricPrompt API:
<https://developer.android.com/reference/androidx/biometric/BiometricPrompt>
- Android Developers — EncryptedSharedPreferences:
<https://developer.android.com/reference/androidx/security/crypto/EncryptedSharedPreferences>
- Android Developers — Network Security Configuration:
<https://developer.android.com/privacy-and-security/security-config>
- Firebase Documentation — Cloud Firestore:
<https://firebase.google.com/docs/firestore>
- Firebase Documentation — Firebase Authentication:
<https://firebase.google.com/docs/auth>
- Firebase Documentation — Firestore Security Rules:
<https://firebase.google.com/docs/firestore/security/get-started>
- Material Design 3 Guidelines: <https://m3.material.io/>

Seguridad y criptografía aplicada

- OWASP Mobile Security Testing Guide (MSTG): <https://owasp.org/www-project-mobile-security-testing-guide/>
- OWASP Mobile Application Security Verification Standard (MASVS) v2.0:
<https://mas.owasp.org/MASVS/>

13. Bibliografía y fuentes consultadas

- OWASP Password Storage Cheat Sheet (2023): https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- OWASP Cryptographic Storage Cheat Sheet (2024): https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html
- OWASP Mobile Top 10 (2024): <https://owasp.org/www-project-mobile-top-10/>
- INCIBE-CERT — Glosario de términos de ciberseguridad: <https://www.incibe.es/incibe-cert/glosario-de-terminos>
- Goodin, D. (2022). «LastPass says hackers have obtained vault data». Ars Technica: <https://arstechnica.com/information-technology/2022/12/lastpass-says-hackers-have-obtained-vault-data-and-a-wealth-of-customer-info/>
- Bitwarden Security Whitepaper: <https://bitwarden.com/help/bitwarden-security-white-paper/>

Gestión de contraseñas y arquitecturas Zero-Knowledge

- 1Password — Security Design White Paper: <https://1passwordstatic.com/files/security/1password-white-paper.pdf>
- Proton Pass — Security Model: <https://proton.me/blog/proton-pass-security-model>
- Khan Academy — Criptografía (en español): <https://es.khanacademy.org/computing/computer-science/cryptography>
- Boneh, D., Shoup, V. — A Graduate Course in Applied Cryptography (versión online gratuita): <https://toc.cryptobook.us/>
- RFC 2898 — PKCS #5: Password-Based Cryptography Specification v2.0: <https://www.rfc-editor.org/rfc/rfc2898>
- RFC 8018 — PKCS #5: Password-Based Cryptography Specification v2.1: <https://www.rfc-editor.org/rfc/rfc8018>

14. Anexos

Anexo A — Estudio sobre cifrado y seguridad criptográfica

Anexo B — Estudio sobre Firebase y arquitectura de base de datos en la nube

Anexo C — Glosario de términos

Término	Definición
AES-256-GCM	Advanced Encryption Standard con clave de 256 bits en modo Galois/Counter Mode. Proporciona cifrado autenticado.
PBKDF2	Password-Based Key Derivation Function 2. Algoritmo para derivar claves criptográficas a partir de contraseñas.
DEK	Data Encryption Key. Clave que cifra los datos del usuario. Nunca se almacena en abierto
KEK	Key Encryption Key. Clave derivada de la contraseña del usuario, usada para cifrar la DEK.
PUK	Personal Unblocking Key. Código de recuperación de acceso generado en el registro.
Zero-Knowledge	Arquitectura en la que el servidor nunca tiene acceso a los datos descifrados
IV / Nonce	Vector de inicialización. Valor aleatorio usado en el cifrado para garantizar que el mismo texto plano produce textos cifrados distintos.
TEE	Trusted Execution Environment. Procesador seguro aislado usado por el Android Keystore.

14. Anexos

Autofill	Sistema de Android que permite a aplicaciones externas rellenar formularios automáticamente.
Vault	Bóveda. Contenedor cifrado con todas las credenciales del usuario.
Zeroización	Proceso de sobrescribir datos sensibles en memoria con ceros para minimizar su exposición.
Firebase	Plataforma de desarrollo de aplicaciones de Google que ofrece servicios de backend gestionados como autenticación, base de datos y funciones en la nube
Firebase Authentication	Servicio de Firebase que gestiona la identidad del usuario. En Clef se usa con los proveedores Google y email/contraseña.
Cloud Firestore	Base de datos NoSQL en la nube de Firebase. En Clef almacena el documento del usuario con el salt, las cajas cifradas, el vault y los metadatos de acceso.
Cloud Functions	Funciones ejecutadas en el servidor de Google sin infraestructura propia. En Clef gestionan las alertas de acceso, la notificación de cambio de contraseña maestra y la eliminación de cuenta.
Plan Blaze	Plan de pago por uso de Firebase. Necesario para usar Cloud Functions, aunque el uso académico se mantiene dentro de los límites gratuitos.
SDK Admin	Biblioteca de Firebase para entornos de servidor. Opera con privilegios elevados y no está sujeta a las reglas de seguridad de Firestore.
Google Cloud Secret	Servicio de Google Cloud para almacenar secretos de forma segura. En Clef guarda la contraseña de la cuenta de correo usada para las alertas.
Reglas de seguridad	Configuración de Firestore que restringe el acceso a los documentos. En Clef garantizan que cada usuario solo puede leer y escribir su propio documento.